

8. Postman Collection Structure

8.1 Overview of Postman Collection Structure

The **Postman Collection Structure** defines how all API requests are organized in a systematic and logical way inside Postman. In this project, the ReqRes API endpoints are grouped into different folders based on their functionality.

This structured approach helps in:

- Easy navigation of APIs
- Better maintainability of test cases
- Reusability of requests
- Clear separation of different API functionalities
- Simulating real-world API project organization used in QA teams

The collection is divided into five main modules:

1. Authentication
 2. Users
 3. Create User
 4. Update User
 5. Delete User
-

8.2 1. Authentication Folder

Purpose

This folder contains APIs related to user authentication. Authentication is the first step in any API system where user identity is verified before accessing other resources.

API Included

Login API

- **Method:** POST
- **Endpoint:** `{{base_url}}/login`

Functionality

- Sends user credentials (email and password)
- Validates user login
- Generates authentication token upon success

Importance in Collection

- Acts as the starting point of API testing
 - Used to simulate real-world login systems
 - Ensures authentication flow is working correctly
-

8.3 2. Users Folder

Purpose

This folder contains APIs related to retrieving user information from the system.

APIs Included

8.3.1 Get Users API

- **Method:** GET
- **Endpoint:** `{{base_url}}/users?page=2`

Functionality

- Retrieves a list of users
- Supports pagination
- Returns multiple user records in JSON format

Validation Points

- Status code should be 200
 - User list should be returned
 - Pagination data should be correct
-

8.3.2 Get Single User API

- **Method:** GET
- **Endpoint:** `{{base_url}}/users/2`

Functionality

- Retrieves details of a specific user using user ID
- Returns user profile information

Validation Points

- Correct user data should be returned
 - Status code should be 200
 - Response structure should be consistent
-

Importance of Users Folder

- Helps test data retrieval operations
 - Validates backend database responses
 - Ensures API supports filtering and fetching mechanisms
-

8.4 3. Create User Folder

Purpose

This folder contains APIs used for creating new user records in the system.

API Included

Create User API

- **Method:** POST
- **Endpoint:** `{{base_url}}/users`

Functionality

- Sends new user details (name, job)
- Creates a new user in the system
- Returns newly created user details

Request Example

```
{
  "name": "Tanhaji",
  "job": "QA Engineer"
}
```

Validation Points

- Status code should be 201
- User ID should be generated
- Response should contain createdAt timestamp

Importance

- Simulates real-world user registration process
 - Validates backend data creation logic
-

8.5 4. Update User Folder

Purpose

This folder contains APIs used for updating existing user information.

APIs Included

8.5.1 PUT Update User API

- **Method:** PUT
- **Endpoint:** `{{base_url}}/users/2`

Functionality

- Fully updates user data
- Replaces existing user information

Validation Points

- Status code should be 200
 - All fields should be updated correctly
-

8.5.2 PATCH Update User API

- **Method:** PATCH
- **Endpoint:** `{{base_url}}/users/2`

Functionality

- Partially updates user data
- Only selected fields are modified

Validation Points

- Only updated fields should change
 - Other data should remain unchanged
 - Status code should be 200
-

Importance of Update Folder

- Ensures API supports both full and partial updates
 - Validates data modification logic
 - Simulates real-world editing operations in applications
-

8.6 5. Delete User Folder

Purpose

This folder contains APIs used for deleting user records from the system.

API Included

Delete User API

- **Method:** DELETE
- **Endpoint:** `{{base_url}}/users/2`

Functionality

- Deletes a specific user based on ID
- Removes user data from system

Validation Points

- Status code should be 204 (No Content)
- No response body should be returned
- Request should complete successfully

Importance

- Validates data deletion functionality
 - Ensures system can handle removal of records safely
 - Simulates real-world account deletion feature
-

8.7 Overall Structure Summary

Folder	Purpose
Authentication	Handles login and token generation
Users	Retrieves user data (all & single user)
Create User	Adds new user to system
Update User	Modifies existing user data
Delete User	Removes user from system

8.8 Final Conclusion

The Postman Collection is designed in a structured and modular format to represent real-world API project organization. Each folder is categorized based on functionality, making it easy to manage, execute, and maintain API test cases.

This structured approach helps in:

- Better test organization
 - Easier debugging
 - Efficient test execution
 - Real-world QA workflow simulation
-